LAPORAN PRAKTIKUM
WEB DESAIN
MODUL 2
GIT DAN GITHUB



Disusun oleh :

Nama	**:**	**Muhammad Ihza Algifari**
NIM	**:**	**20250910039**
Kelas	**:**	**SINFC 02 2025**
Program Studi	**:**	**Sistem Informasi**

FAKULTAS ILMU KOMPUTER
UNIVERSITAS KUNINGAN
2026

KATA PENGANTAR

Laporan ini dibuat untuk pelaksanaan praktikum Modul 2 mengenai Git dan GitHub yang merupakan bagian dari mata kuliah Praktikum Desain Web pada Program Studi Sistem Informasi, Fakultas Ilmu Komputer, Universitas Kuningan. Praktikum ini bertujuan agar mahasiswa memahami konsep version control, membuat akun GitHub, memasang dan mengonfigurasi Git, menggunakan perintah dasar Git seperti init, add, commit, branch, merge, dan push, serta mengelola proyek secara kolaboratif melalui remote repository.

Dalam laporan ini, saya menuliskan jawaban pre-test, langkah-langkah praktikum yang disesuaikan dengan repository GitHub yang saya buat (Praktikum-Modul-1), jawaban post-test, serta pengerjaan latihan/tugas beserta kesimpulannya. Saya menyadari bahwa laporan ini masih memiliki kekurangan karena keterbatasan pengetahuan dan kemampuan saya, sehingga kritik dan saran yang membangun sangat saya harapkan. Terima kasih saya ucapkan kepada dosen pengampu dan asisten praktikum yang telah membimbing selama pelaksanaan praktikum dan penyusunan laporan ini. Semoga laporan ini bermanfaat.

Kuningan, 4 November 2026

Muhammad Ihza Algifari

DAFTAR ISI

KATA PENGANTAR ..... **ii**

DAFTAR ISI ..... **iii**

BAB I PRETEST **1**

BAB II PRAKTIKUM **3**

A. Membuat Akun GitHub 3

B. Instalasi dan Konfigurasi Git 3

C. Mengelola Proyek dengan Git 4

D. Menghubungkan ke GitHub dan Push Perubahan 5

BAB III POST-TEST **9**

BAB IV LATIHAN/TUGAS **9**

BAB V KESIMPULAN **16**

BAB I
PRETEST

1. Apa tujuan dari menggunakan version control dalam pengembangan perangkat lunak?
2. Jelaskan perbedaan antara git add dan git commit!
3. Bagaimana cara membuat branch baru di Git?

Jawaban:

1. Tujuan penggunaan version control

Version control (sistem kontrol versi) digunakan untuk melacak setiap perubahan pada file proyek sehingga seluruh riwayat pekerjaan tersimpan dan dapat dikelola. Tujuan utamanya:

- Merekam riwayat perubahan: setiap perubahan tercatat lengkap dengan siapa yang mengubah, kapan, dan apa alasannya (pesan commit).
- Mengembalikan versi sebelumnya: jika terjadi kesalahan atau bug, proyek dapat dikembalikan ke kondisi yang masih berfungsi.
- Memudahkan kolaborasi: beberapa orang dapat mengerjakan proyek yang sama tanpa saling menimpa pekerjaan.
- Bekerja paralel dengan branch: fitur baru dapat dikembangkan terpisah dari kode utama yang stabil.
- Menjadi cadangan (backup): saat dihubungkan ke remote repository seperti GitHub, proyek tersimpan di cloud dan dapat diakses dari mana saja.

2. Perbedaan git add dan git commit

Git memiliki tiga area kerja: working directory (tempat file diedit), staging area (kumpulan perubahan yang disiapkan), dan repository (riwayat commit).

Perintah git add dan git commit bekerja pada dua tahap berbeda.

git add

Fungsi: Memasukkan file atau perubahan dari working directory ke staging area.

Hasil: File berstatus "Changes to be committed"; belum tersimpan di riwayat.

Contoh: git add Latihan_01_01.html

git commit

Fungsi: Merekam isi staging area secara permanen ke repository sebagai satu snapshot (commit).

Hasil: Terbentuk commit baru dengan ID (hash), penulis, waktu, dan pesan.

Contoh: git commit -m "Modul 1"

3. Cara membuat branch baru di Git

Branch baru dibuat dengan perintah git branch diikuti nama branch, kemudian berpindah ke branch tersebut. Keduanya dapat digabung dengan satu perintah.

git branch fitur-baru

Perintah tersebut digunakan untuk membuat branch fitur-baru.

git switch fitur-baru

Perintah tersebut digunakan untuk berpindah ke branch fitur-baru.

git switch -c fitur-baru

Perintah tersebut digunakan untuk membuat sekaligus berpindah ke branch fitur-baru (cara singkat).

git checkout -b fitur-baru

Perintah tersebut merupakan alternatif cara lama yang setara.

git branch

Perintah tersebut digunakan untuk melihat daftar branch. Tanda * menunjukkan branch aktif.

BAB II
PRAKTIKUM

Praktikum dilaksanakan mengikuti urutan pada Modul 2 dan disesuaikan dengan akun serta repository GitHub milik saya, yaitu akun MuhammadIhzaAlgifari dan repository Praktikum-Modul-1.
Berbeda dengan contoh pada modul (repository pweb1 dengan folder Modul_01), seluruh file Modul 1 pada repository saya tersimpan langsung di root repository, sehingga perintah git add disesuaikan tanpa awalan folder.

A. Membuat Akun GitHub

1. Membuka situs GitHub melalui browser di alamat https://github.com.
2. Mengklik tombol Sign up di pojok kanan atas.
3. Mengisi email, password, dan username (MuhammadIhzaAlgifari), lalu mengklik Continue.
4. Menyelesaikan verifikasi puzzle "Verify your account" yang ditampilkan GitHub.
5. Membuka email dan mengambil kode verifikasi yang dikirim oleh GitHub.
6. Memasukkan kode verifikasi pada kotak isian, kemudian login ke akun yang telah berhasil dibuat.
7. Melengkapi profil dan memilih paket akun Free sesuai petunjuk modul.

Hasilnya, akun GitHub dengan username MuhammadIhzaAlgifari dapat digunakan untuk membuat repository, seperti terlihat pada Gambar 2.1 di bagian akhir Bab ini.

B. Instalasi dan Konfigurasi Git

1. Mengunduh Git dari https://git-scm.com dan menginstalnya di Windows mengikuti petunjuk instalasi.
2. Membuka Visual Studio Code, lalu membuka terminal melalui View > Terminal atau tombol Ctrl + `.
3. Memastikan Git terpasang dengan benar dengan perintah berikut. Terminal menampilkan nomor versi Git yang terpasang.

git --version

4. Mengonfigurasi nama pengguna dan email yang akan tercatat pada setiap commit.

git config --global user.name "Muhammad Ihza Algifari"

git config --global user.email "20250910039@uniku.ac.id"

Nama dan email ini muncul sebagai Author pada riwayat commit, sebagaimana terlihat pada hasil git log di bagian berikutnya.

C. Mengelola Proyek dengan Git

Folder proyek berisi file hasil praktikum Modul 1, yaitu Latihan_01_01.html sampai Latihan_01_05.html, POST_TEST_01.html, LATIHAN_TUGAS_01.html, dan README.md.

1. Menginisialisasi repository Git baru

Menginisialisasi repository Git baru pada folder proyek melalui terminal Visual Studio Code. Branch awal dinamai main.

git init

2. Memeriksa status repository

Seluruh file masih berstatus untracked (belum dilacak Git).

git status

Hasil:

On branch main

No commits yet

Untracked files:

(use "git add <file>..." to include in what will be committed)

LATIHAN_TUGAS_01.html

Latihan_01_01.html

Latihan_01_02.html

Latihan_01_03.html

Latihan_01_04.html

Latihan_01_05.html

POST_TEST_01.html

README.md

nothing added to commit but untracked files present (use "git add" to track)

3. Menambahkan file ke staging area

Pada contoh modul hanya satu file yang ditambahkan; pada repository ini semua file Modul 1 ditambahkan sekaligus.

git add Latihan_01_01.html

Perintah tersebut digunakan untuk menambahkan satu file (contoh pada modul).

git add .

Perintah tersebut digunakan untuk menambahkan seluruh file Modul 1.

4. Memeriksa kembali status repository

Semua file kini berstatus new file pada bagian "Changes to be committed", artinya siap di-commit.

git status

Hasil:

On branch main

No commits yet

Changes to be committed:

(use "git rm --cached <file>..." to unstage)

new file: LATIHAN_TUGAS_01.html

new file: Latihan_01_01.html

new file: Latihan_01_02.html

new file: Latihan_01_03.html

new file: Latihan_01_04.html

new file: Latihan_01_05.html

new file: POST_TEST_01.html

new file: README.md

5. Merekam perubahan

Merekam perubahan dari staging area ke repository dengan pesan commit "Modul 1".

git commit -m "Modul 1"

6. Melihat riwayat commit

Melihat riwayat commit yang terbentuk beserta ringkasan file di dalamnya.

git log

Hasil:

commit 1fd4bfb0f525ff41599abbeb377e11124b43f5aa

Author: Muhammad Ihza Algifari <20250910039@uniku.ac.id>

Date: Thu Oct 29 11:54:22 2026 +0700

   Modul 1

Kemudian melihat ringkasan file yang terdapat dalam commit.

git show --stat --format=

Hasil:

LATIHAN_TUGAS_01.html | 89 +++++++++++++++

Latihan_01_01.html | 17 +++

Latihan_01_02.html | 57 +++++++++

Latihan_01_03.html | 37 ++++++

Latihan_01_04.html | 55 +++++++++

Latihan_01_05.html | 78 ++++++++++++

POST_TEST_01.html | 328

++

README.md | Bin 0 -> 212 bytes

8 files changed, 661 insertions(+)

Hasilnya, satu commit dengan ID 1fd4bfb berisi 8 file dan 661 baris tambahan berhasil dibuat di repository lokal.

D. Menghubungkan ke GitHub dan Push Perubahan

1. Membuat repository baru di GitHub

Membuat repository baru di GitHub dengan nama Praktikum-Modul-1 (visibility Public). Nama ini menggantikan contoh pweb1 pada modul.

2. Menghubungkan repository lokal ke remote repository di GitHub

git remote add origin https://github.com/MuhammadIhzaAlgifari/Praktikum-Modul-1.git

3. Memastikan nama branch utama adalah main

git branch -M main

4. Mengirim seluruh commit ke GitHub

Mengirim seluruh commit pada branch main ke remote repository di GitHub. Opsi -u menyimpan origin main sebagai tujuan default untuk push dan pull berikutnya.

git push -u origin main

5. Mengamati hasilnya

Pengaturan remote dan branch dapat diperiksa dengan perintah berikut.

git remote -v

Hasil:

origin https://github.com/MuhammadIhzaAlgifari/Praktikum-Modul-1.git (fetch)

origin https://github.com/MuhammadIhzaAlgifari/Praktikum-Modul-1.git (push)

Kemudian memeriksa status branch.

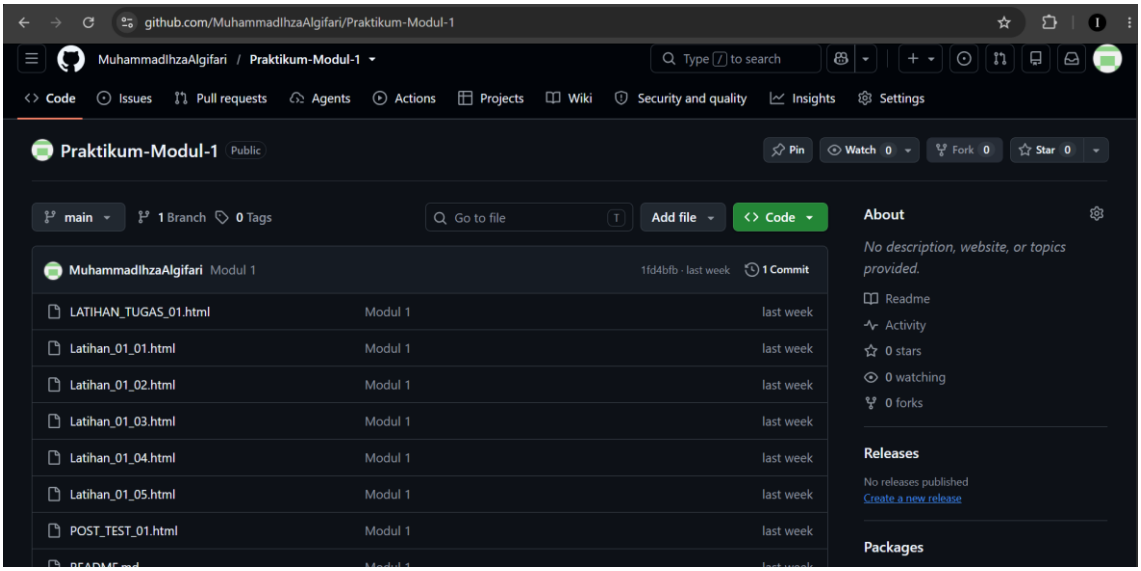
git status -sb

Hasil:

main...origin/main

Status main...origin/main tanpa selisih menunjukkan branch lokal sudah sama dengan branch di GitHub.

Pada halaman repository Praktikum-Modul-1 di GitHub (Gambar 2.1) terlihat branch main, 1 commit ("Modul 1", ID 1fd4bfb), dan seluruh file Modul 1. Proyek kini tersimpan di cloud dan dapat diakses dari mana saja.



Gambar 2.1 Repository Praktikum-Modul-1 pada akun GitHub MuhammadIhzaAlgifari setelah push

BAB III
POST-TEST

1. Apa fungsi dari git merge dalam workflow Git?
2. Jelaskan cara mengatasi konflik saat melakukan merge!
3. Tulis langkah-langkah untuk menambahkan file baru ke repository Git dan melakukan commit.

Jawaban:

1. **Fungsi git merge**

   git merge menggabungkan perubahan dari satu branch ke branch lain, biasanya dari branch fitur ke branch utama (main). Dengan merge, pekerjaan yang dikembangkan terpisah dapat diintegrasikan kembali menjadi satu riwayat.Ada dua hasil umum: fast-forward (jika main tidak berubah sejak branch dibuat, penunjuk branch cukup digeser maju) atau merge commit (jika kedua branch sama-sama punya commit baru).

   Berikut contoh pada repository percobaan:

git switch main

git merge fitur-biodata

Hasil:

Updating 692e73f..e621183

Fast-forward

index.html | 1 +

1 file changed, 1 insertion(+)

2. Cara mengatasi konflik saat merge

Konflik terjadi ketika dua branch mengubah baris yang sama pada file yang sama sehingga Git tidak dapat memutuskan versi mana yang dipakai.

Contoh: branch main mengubah baris hobi menjadi "Coding", sedangkan branch konflik-a mengubahnya menjadi "Membaca".

git merge konflik-a

Hasil:

Auto-merging index.html

CONFLICT (content): Merge conflict in index.html

Automatic merge failed; fix conflicts and then commit the result.

Langkah penyelesaiannya:

1. Menjalankan git status untuk melihat file yang konflik (ditandai UU atau "both modified").
2. Membuka file tersebut. Git menandai bagian konflik seperti berikut:

<<<<<<< HEAD

Hobi: Coding

=======

Hobi: Membaca

>>>>>>> konflik-a

3. Mengedit file: memilih salah satu versi, atau menggabungkan keduanya, lalu menghapus penanda >>>>>>.

Hasil akhir pada contoh:

Hobi: Membaca dan Coding

4. Menandai konflik telah selesai dengan git add index.html.
5. Menyelesaikan merge dengan git commit. Hasilnya terbentuk merge commit:

git log --oneline --graph --all

Hasil:

```
*   3f80a64 Merge branch konflik-a: gabung hobi
|\
| * c470674 Hobi: Membaca
* | a0edf32 Hobi: Coding
|/
* e621183 Tambah program studi
* 692e73f Halaman awal biodata
```

Jika ingin membatalkan proses merge, gunakan git merge --abort sehingga kondisi kembali seperti sebelum merge.

3. Langkah menambahkan file baru ke repository dan commit

1. Membuat file baru di folder repository, misalnya Latihan_02_01.html, menggunakan Visual Studio Code.
2. Memeriksa status dengan git status. File baru muncul sebagai untracked.
3. Menambahkan file ke staging area dengan git add Latihan_02_01.html (atau git add . untuk semua perubahan).
4. Memeriksa kembali dengan git status. File berstatus new file dan siap di-commit.
5. Merekam perubahan dengan git commit -m "Menambahkan Latihan_02_01.html".
6. (Opsional) Mengirim commit ke GitHub dengan git push.

Perintah yang digunakan:

```
git status
git add Latihan_02_01.html
git commit -m "Menambahkan Latihan_02_01.html"
git push
```

BAB IV

LATIHAN/TUGAS

1. Membuat repository Git untuk proyek web dan mendokumentasikan prosesnya

Proyek web yang digunakan adalah kumpulan hasil praktikum HTML (Modul 1) yang disimpan pada repository Praktikum-Modul-1.
Informasi repository:
Nama repository: Praktikum-Modul-1
URL: https://github.com/MuhammadIhzaAlgifari/Praktikum-Modul-1
Visibility / branch: Public / main
Commit: 1fd4bfb, pesan "Modul 1", 29 Oktober 2026
Isi: 8 file (7 file HTML dan README.md), 661 baris tambahan
Dokumentasi proses pembuatan, commit, dan push dirangkum sebagai berikut (detail dan keluaran perintah ada pada Bab II).

1. Inisialisasi repository
Perintah:
git init
Hasil: Repository lokal dibuat, branch main.

2. Cek status
Perintah:
git status
Hasil: 8 file berstatus untracked.

3. Staging
Perintah:
git add .
Hasil: 8 file masuk staging area.

4. Commit
Perintah:
git commit -m "Modul 1"
Hasil: Commit 1fd4bfb terbentuk.

5. Hubungkan remote
Perintah:
git remote add origin <URL>
Hasil: Remote origin terdaftar.

6. Set branch utama
Perintah:
git branch -M main
Hasil: Branch bernama main.

7. Push
Perintah:
git push -u origin main

2. Workflow Git dengan branch untuk fitur baru dan integrasi ke branch utama

Workflow yang dirancang adalah feature branch workflow: branch main selalu berisi versi stabil, sedangkan setiap fitur baru dikerjakan pada branch tersendiri lalu digabung kembali ke main setelah selesai. Sebagai contoh, fitur baru yang dikembangkan adalah halaman biodata (fitur-biodata).

1. **Sinkronkan main**: Gunakan perintah git switch main dan git pull untuk memastikan *branch main* lokal sama dengan yang ada di GitHub.
2. **Buat branch fitur**: Gunakan perintah git switch -c fitur-biodata untuk membuat sekaligus berpindah ke *branch* fitur baru.
3. **Kerjakan fitur**: Kerjakan fitur dan lakukan *commit* berkala pada *branch* fitur menggunakan perintah git add . lalu git commit -m "Tambah halaman biodata".
4. **Kirim branch**: Gunakan perintah git push -u origin fitur-biodata untuk mencadangkan *branch* fitur di GitHub.
5. **Integrasi ke main**: Beralih ke *branch main* dengan git switch main lalu gabungkan menggunakan git merge fitur-biodata (selesaikan konflik jika terjadi seperti Bab III).
6. **Publikasikan**: Gunakan perintah git push origin main agar *branch main* di GitHub terisi fitur baru.
7. **Bersihkan**: Gunakan perintah git branch -d fitur-biodata untuk menghapus *branch* fitur yang sudah selesai digabungkan.

Pada langkah 4-5, integrasi juga dapat dilakukan lewat Pull Request di GitHub agar perubahan dapat ditinjau sebelum digabung ke main. Contoh hasil merge branch fitur pada repository percobaan dapat dilihat pada Bab III (git merge fitur-biodata berjalan fast-forward).

3. Menyimpan seluruh praktikum modul sebelumnya ke repository Git

Seluruh hasil praktikum Modul 1 (HTML) sudah tersimpan pada repository Praktikum-Modul-1 sebagaimana tabel berikut.

1. Pada bagian Latihan 1, berkas Latihan_01_01.html memuat penerapan tag font (size) dan garis pembatas hr yang menyajikan informasi profil FIK UNIKU.
2. Pada bagian Latihan 2, berkas Latihan_01_02.html berisi penggunaan elemen *heading* serta struktur daftar berurut (*ordered list* / ol).
3. Pada bagian Latihan 3, berkas Latihan_01_03.html memuat penyisipan gambar (img) dan penerapan daftar tidak berurut (*unordered list* / ul).
4. Pada bagian Latihan 4, berkas Latihan_01_04.html berisi pembuatan dan penyusunan struktur tabel HTML.

5. Pada bagian Latihan 5, berkas Latihan_01_05.html memuat formulir data diri yang menggunakan berbagai elemen input seperti input, radio, textarea, dan select.
6. Pada bagian Post-test, berkas POST_TEST_01.html menyajikan informasi lengkap mengenai profil Fakultas Ilmu Komputer UNIKU beserta program studinya.
7. Pada bagian Latihan/Tugas, berkas LATIHAN_TUGAS_01.html berisi rancangan *landing page* untuk penayangan biodata diri.
8. Pada bagian Dokumentasi, berkas README.md berfungsi sebagai berkas dokumentasi utama penjelas isi repositori.

Untuk modul berikutnya (misalnya Modul 3 CSS), file baru cukup ditambahkan ke folder repository, lalu disimpan dengan alur git add, git commit, dan git push seperti pada Bab II sehingga setiap modul memiliki riwayat commit sendiri.

BAB V
KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, saya memahami bahwa Git adalah sistem version control yang merekam riwayat perubahan proyek sehingga pekerjaan dapat dikembalikan, dikembangkan secara paralel, dan dikerjakan bersama.

Saya telah membuat akun GitHub, memasang dan mengonfigurasi Git, serta menggunakan perintah dasar git init, status, add, commit, remote, branch, merge, dan push.

Alur kerja Git terdiri dari working directory, staging area, dan repository: git add menyiapkan perubahan, git commit merekamnya, dan git push mengirimnya ke GitHub.

Branch memungkinkan pengembangan fitur tanpa mengganggu versi utama, dan konflik saat merge dapat diselesaikan dengan mengedit penanda konflik lalu melakukan commit.

Seluruh hasil praktikum Modul 1 kini tersimpan di repository Praktikum-Modul-1 sehingga terdokumentasi dengan baik dan dapat diakses dari mana saja.